

TensorFlow / Keras 개요

정의

주요 모듈

tf.keras.datasets

tf.keras.Sequential

add

compile

fit

evaluate

predict

save

tf.keras.layers

tf.keras.layers.Flatten

tf.keras.layers.Dense

tf.keras.layers 예제

실습

정의

- 신경망 구현을 위해 python code로 작성된 high-level API
- 사용자가 손쉽게 딥 러닝을 구현할 수 있도록 도와주는 상위 레벨의 인터페이스로 딥러닝을 쉽게 구현할 수 있음
- API 공식 문서 : https://www.tensorflow.org/api_docs/python/tf/keras

주요 모듈

tf.keras.datasets

- [api_docs 링크](#)
- fashion_mnist 뿐만 아니라 boston_housing, imdb 등 여러 데이터를 갖고 있음

```
fashion_mnist = keras.datasets.fashion_mnist
```

```
(train_images, train_labels), (test_images, test_labels) = fashion_mnist.load_data()
```

- 각 module은 load_data() 함수를 가집니다.
- 해당 함수는 (x_train, y_train), (x_test, y_test) 형태로 Numpy array를 요소로 갖는 Tuple을 return함

tf.keras.Sequential

- [api_docs 링크](#)
- 입력층, 은닉층, 출력층을 구성하기 위해 Sequential()을 사용, 모델을 구축하는데 쓰이는 module
- Sequential()을 model로 선언한 뒤에 model.add()라는 코드를 통해 층을 단계적으로 추가함
- Sequential은 Model 을 상속함

add

layer를 추가하는 함수

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential()
model.add(...) # 층 추가
model.add(...) # 층 추가
model.add(...) # 층 추가

# Embedding()을 통해 생성하는 임베딩 층(embedding layer)을 추가
model = Sequential()
model.add(Embedding(vocab_size, output_dim, input_length))

# fully-connected layer를 추가
model = Sequential()
model.add(Dense(1, input_dim=3, activation='relu'))
```

compile

구축한 모델에 필요한 추가적인 설정을 부여하는 함수

```
model.compile(
    optimizer='rmsprop', loss=None, metrics=None, loss_weights=None,
```

```

sample_weight_mode=None, weighted_metrics=None, target_tensors=None,
e,
    distribute=None, **kwargs
)

# 예제
model = Sequential()
model.add(Embedding(vocab_size, embedding_dim))
model.add(SimpleRNN(hidden_units))
model.add(Dense(1, activation='sigmoid'))
model.compile(optimizer='rmsprop', loss='binary_crossentropy', metrics=['acc'])

```

- **optimizer** : 비용 함수의 그래디언트에 따라 가중치를 학습시키는 방법 지정
 - adam, adagrad, SGD 등이 있음
- **loss** : 훈련하는 동안 모델의 손실 오류를 계산할 방법 지정
 - MeanSquareError, SparseCategoricalEntropy 등이 있음
- **metrics** : 훈련 단계와 테스트 단계를 모니터링하기 위한 지표 지정
 - accuracy, mse 등이 있음

문제 유형	손실 함수명	출력층의 활성화 함수명	참고 실습
회귀 문제	mean_squared_error	-	선형 회귀 실습
다중 클래스 분류	categorical_crossentropy	소프트맥스	로이터 뉴스 분류하기
다중 클래스 분류	sparse_categorical_crossentropy	소프트맥스	양방향 LSTM을 이용한 품사 태깅
이진 분류	binary_crossentropy	시그모이드	IMDB 리뷰 감성 분류하기

[그림-1] 대표적으로 사용되는 손실 함수와 활성화 함수의 조합

- **sparse_categorical_crossentropy**는 **categorical_crossentropy**와 동일하게 다중 클래스 분류에서 사용하지만, 레이블을 원-핫 인코딩하지 않고 정수 인코딩 된 상태에서 수행 가능하다는 점이 차이점

fit

모델이 오차로부터 매개 변수를 업데이트 시키는 과정으로 모델의 훈련을 시작함

```
model.fit(
    x=None, y=None, batch_size=None, epochs=1, verbose=1, callbacks=None,
    validation_split=0.0, validation_data=None, shuffle=True,
    class_weight=None,
    sample_weight=None, initial_epoch=0, steps_per_epoch=None,
    validation_steps=None, validation_freq=1, max_queue_size=10, workers=1,
    use_multiprocessing=False, **kwargs
)
```

- x : train_images
- y : train_labels
- batch_size : 배치 크기. 기본값은 32
 - 미니 배치 경사 하강법을 사용하고 싶지 않을 경우에는 batch_size=None
- epochs : 훈련을 반복할 epoch 수를 지정
- verbose : 학습 진행 상황 출력 여부 지정

evaluate

테스트 데이터를 통해 학습한 모델에 대한 정확도를 평가

```
model.evaluate(
    x=None, y=None, batch_size=None, verbose=1, sample_weight=None, steps=None,
    callbacks=None, max_queue_size=10, workers=1, use_multiprocessing=False
)
```

- x : test_images
- y : test_labels
- verbose : 학습 중 출력되는 문구가 어떤 빈도로 출력될지 지정
 - 0 = silent, 1 = progress bar, 2 = single line

predict

임의의 입력에 대한 모델의 출력값을 확인

```
model.predict(
    x, batch_size=None, verbose=0, steps=None, callbacks=None,
    max_queue_size=10,
    workers=1, use_multiprocessing=False
)
```

- x : 예측에 사용할 데이터
- 사용가능한 data type : numpy array, tensorflow tensor, tf.data 등

save

인공 신경망 모델을 hdf5 파일에 저장

```
model.save("model_name.h5")
```

load_model

저장해둔 모델을 불러옴

```
from tensorflow.keras.models import load_model
model = load_model("model_name.h5")
```

tf.keras.layers

모델을 구성하는 여러가지 층을 정의하는 모듈로 Flatten, Dense 등이 있음

- [api_docs 링크](#)

tf.keras.layers.Flatten

입력을 평탄화(1차원으로 변환)

- 배치 크기에는 영향을 주지 않음
- [api_docs 링크](#)

```
tf.keras.layers.Flatten(
    data_format=None, **kwargs
)
```

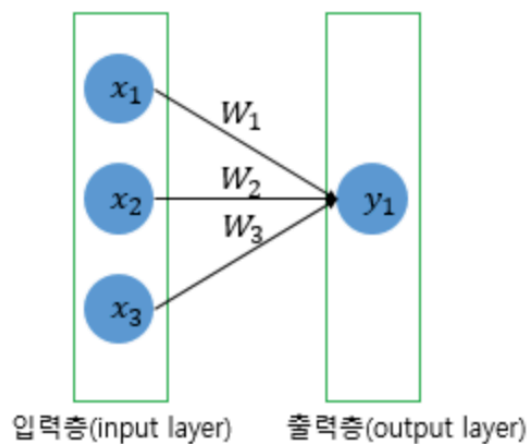
tf.keras.layers.Dense

완전 연결층 생성

- [api_docs 링크](#)

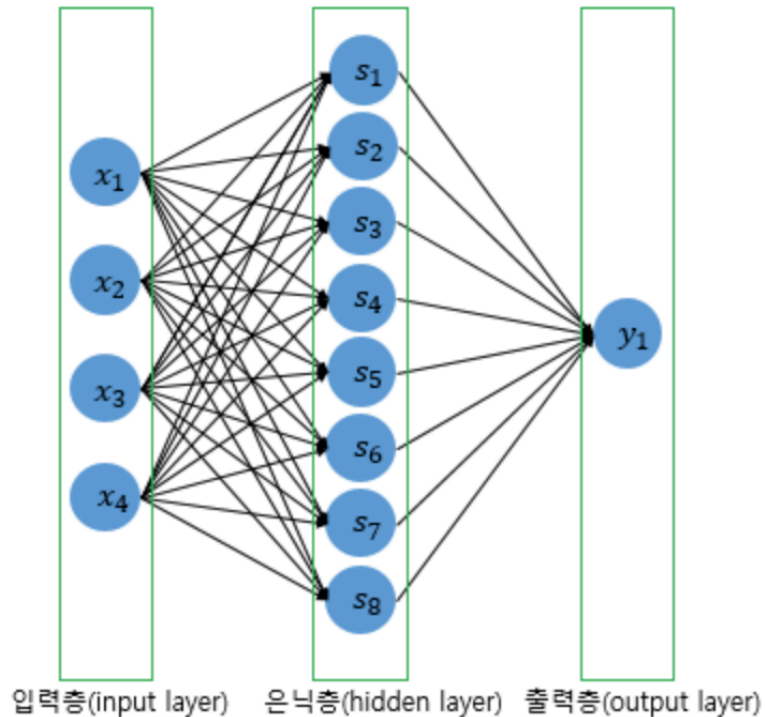
```
model = Sequential()  
model.add(Dense(1, input_dim=3, activation='relu'))
```

- 첫번째 인자 : 출력 뉴런의 수
- input_dim : 입력 뉴런의 수 (입력의 차원)
- activation : 활성화 함수
 - linear : 디폴트 값으로 별도 활성화 함수 없이 입력 뉴런과 가중치의 계산 결과 그대로 출력
 - sigmoid : 이진 분류 문제에서 출력층에 주로 사용되는 활성화 함수
 - softmax : 셋 이상의 선택지 중 하나를 택하는 다중 클래스 분류 문제에서 출력층에 주로 사용되는 활성화 함수
 - relu : 은닉층에 주로 사용되는 활성화 함수



[예제-1]의 시각화

```
model = Sequential()  
model.add(Dense(8, input_dim=4, activation='relu'))  
model.add(Dense(1, activation='sigmoid')) # 출력층
```



[예제-2]의 시각화

tf.keras.layers 예제

- 입력: 28x28 크기의 이미지 (예: MNIST 손글씨 데이터)
- 출력: 10개 클래스(숫자 0~9)에 대한 확률

```
model = tf.keras.models.Sequential([
    Flatten(input_shape = (28, 28)),
    Dense(600, activation = 'relu'),
    Dense(500, activation = 'relu'),
    Dense(10, activation = 'softmax')
])
```

- Flatten(input_shape=(28, 28)): 입력 이미지를 1차원 벡터로 변환
 - 예를 들어, 28x28 픽셀 이미지를 784개의 값으로 펼칩니다. 배치 크기에는 영향을 주지 않음
- Dense(600, activation='relu'): 600개의 뉴런을 가진 완전 연결(은닉)층
 - 활성화 함수로 ReLU를 사용해 비선형성 추가
- Dense(500, activation='relu'): 500개의 뉴런을 가진 두 번째 은닉층
 - 활성화 함수로 ReLU를 사용해 비선형성 추가

- Dense(10, activation='softmax'): 10개의 뉴런을 가진 출력층
 - softmax 활성화 함수로 각 클래스(예: 0~9 숫자)에 대한 확률을 출력

실습

[https://github.com/cserock/colab-examples/blob/main/04_TensorFlow_Keras_%EC%98%88%EC%A0%9C\(1\).ipynb](https://github.com/cserock/colab-examples/blob/main/04_TensorFlow_Keras_%EC%98%88%EC%A0%9C(1).ipynb)

[https://github.com/cserock/colab-examples/blob/main/04_TensorFlow_Keras_%EC%98%88%EC%A0%9C\(2\).ipynb](https://github.com/cserock/colab-examples/blob/main/04_TensorFlow_Keras_%EC%98%88%EC%A0%9C(2).ipynb)